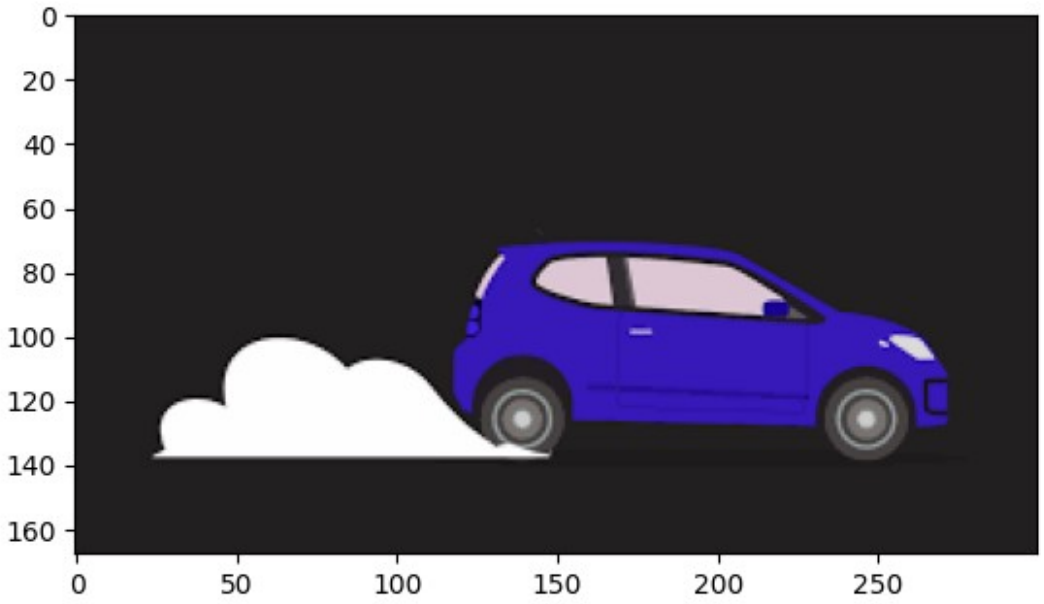
```
import cv2 as cv
```

Practical 1

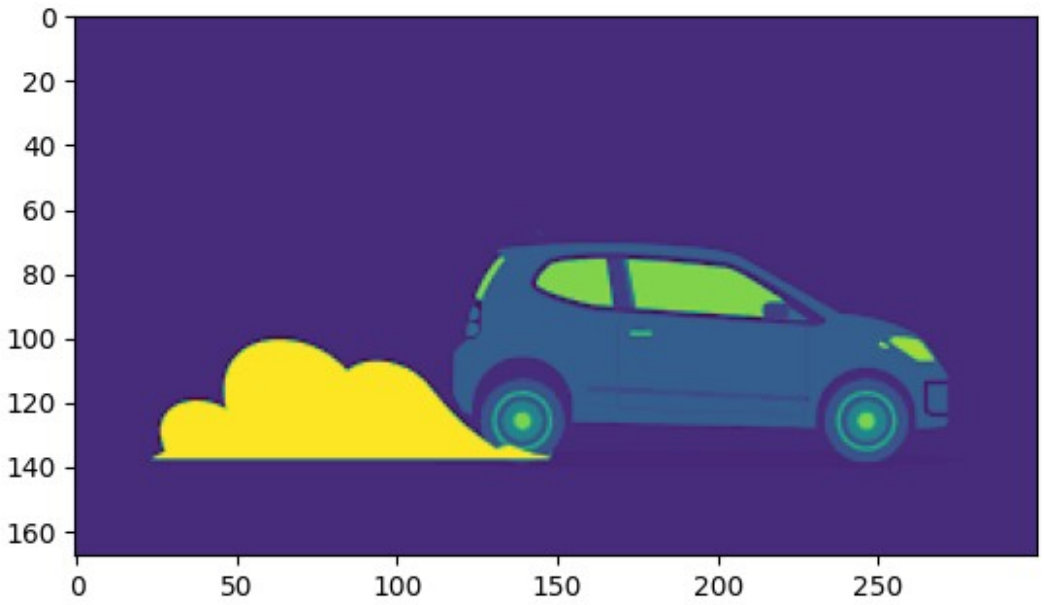
```
import matplotlib.pyplot as plt
import cv2
img = cv2.imread("download.png")
plt.imshow(img)
```

```
<matplotlib.image.AxesImage at 0x14fd13ca8d0>
```



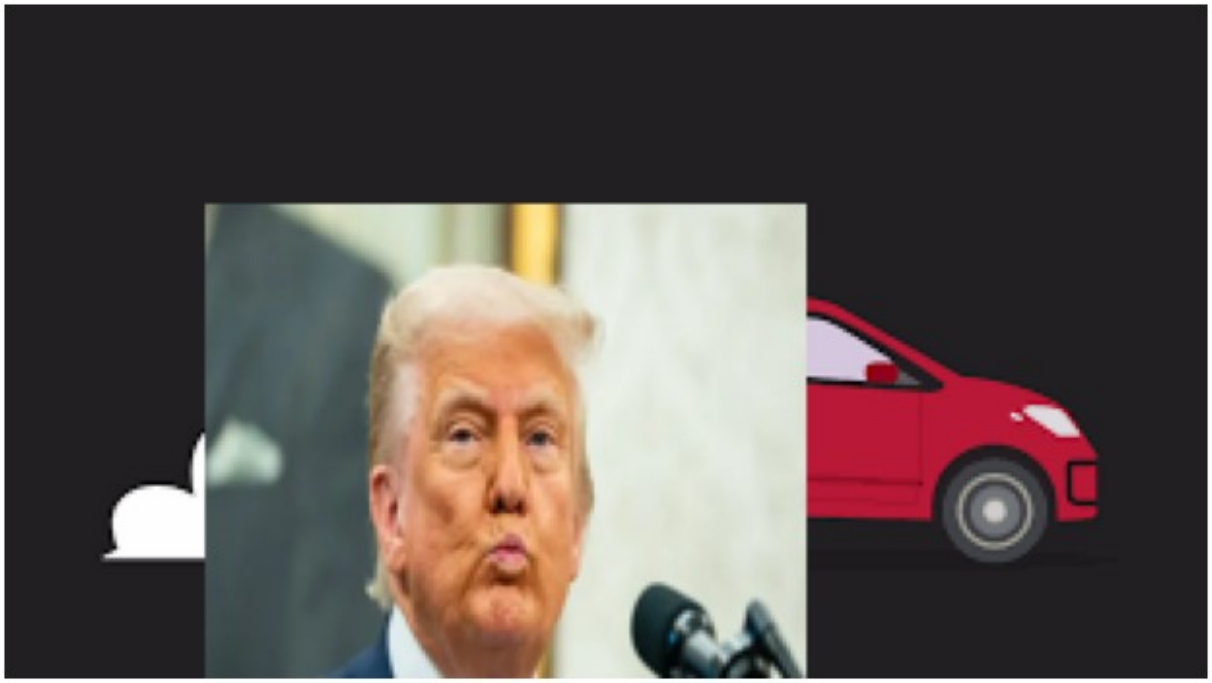

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
plt.imshow(gray)
```

```
<matplotlib.image.AxesImage at 0x14fd14377d0>
```



Practical 2

```python
from PIL import Image
import matplotlib.pyplot as plt

# Open images
background = Image.open("download.png").convert("RGBA")
overlay = Image.open("img.png").convert("RGBA")

# Resize overlay to a smaller size
overlay = overlay.resize((150, 150))  # Adjust as needed

# Position where overlay should appear
x = 50
y = 50

# Create a copy of background
result = background.copy()

# Paste overlay on top
result.paste(overlay, (x, y))

# Display result
plt.figure(figsize=(8, 8))
plt.imshow(result)
plt.axis("off")
plt.show()

# Save result
result.save("output.png")
```



Practical 3

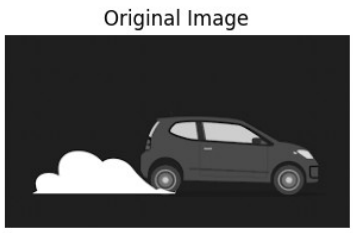
```python
from PIL import Image
import matplotlib.pyplot as plt

# Open image
img = Image.open("download.png").convert("RGBA")

# Set transparency level
# 255 = fully visible
# 128 = 50% transparent
# 64  = 75% transparent
alpha_value = 100

# Apply same alpha to all pixels
img.putalpha(alpha_value)

# Save transparent image
img.save("transparent_img.png")

# Display
plt.imshow(img)
plt.axis("off")
plt.show()
```

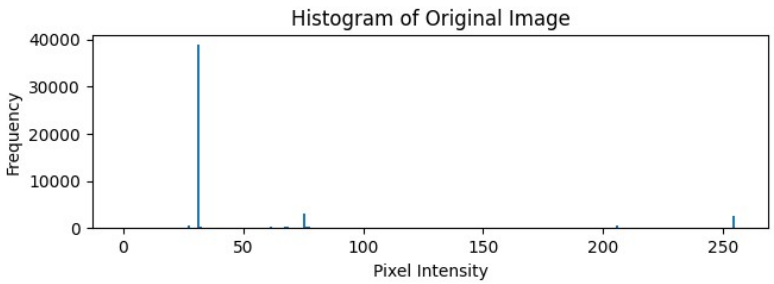
Practical 4

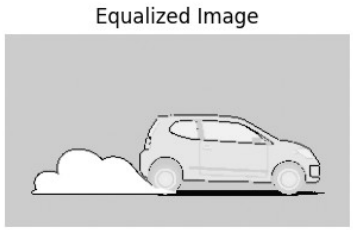
```
import cv2
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("download.png", 0)

# Histogram of original image
plt.figure(figsize=(12,5))

plt.subplot(2,2,1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis('off')

plt.subplot(2,2,2)
plt.hist(img.ravel(), 256, [0,256])
plt.title("Histogram of Original Image")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

# Histogram Equalization
equalized_img = cv2.equalizeHist(img)

plt.subplot(2,2,3)
plt.imshow(equalized_img, cmap='gray')
plt.title("Equalized Image")
plt.axis('off')

plt.subplot(2,2,4)
plt.hist(equalized_img.ravel(), 256, [0,256])
plt.title("Histogram After Equalization")
```

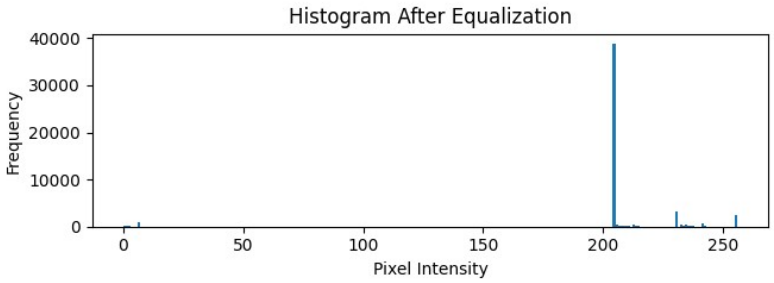
```
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# Save equalized image
cv2.imwrite("equalized_image.jpg", equalized_img)
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_17888\2303394487.py:16:
MatplotlibDeprecationWarning: Passing the range parameter of hist()
positionally is deprecated since Matplotlib 3.10; the parameter will
become keyword-only in 3.12.
  plt.hist(img.ravel(), 256, [0,256])
C:\Users\Admin\AppData\Local\Temp\ipykernel_17888\2303394487.py:30:
MatplotlibDeprecationWarning: Passing the range parameter of hist()
positionally is deprecated since Matplotlib 3.10; the parameter will
become keyword-only in 3.12.
  plt.hist(equalized_img.ravel(), 256, [0,256])
```










```
True
```

Practical 5

```
from PIL import Image

# Open the image
img = Image.open("download.png")

# Convert to RGB (required for JPG if image has transparency)
img = img.convert("RGB")
```

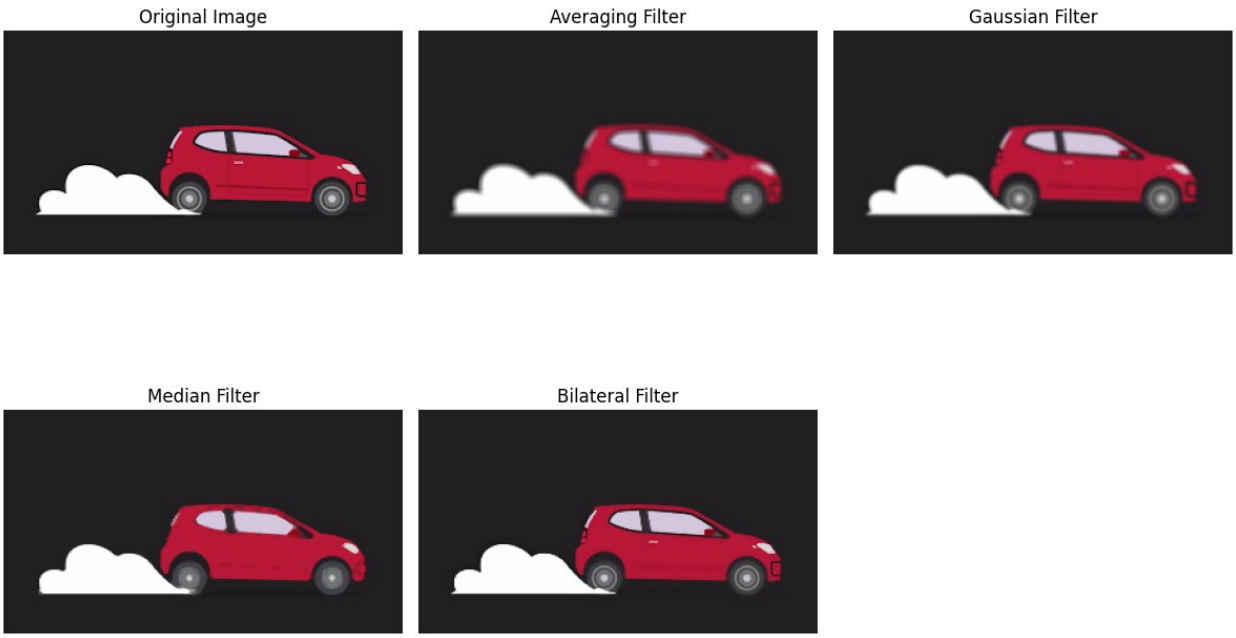
```
# Save as JPG
img.save("output.jpg", "JPEG")

print("Image converted to JPG successfully!")
```

```
Image converted to JPG successfully!
```

Practical 6

```
import cv2
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("download.png")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Apply smoothing filters

# 1. Averaging Filter
avg_filter = cv2.blur(img, (5, 5))

# 2. Gaussian Filter
gaussian_filter = cv2.GaussianBlur(img, (5, 5), 0)

# 3. Median Filter
median_filter = cv2.medianBlur(img, 5)

# 4. Bilateral Filter
bilateral_filter = cv2.bilateralFilter(img, 9, 75, 75)

# Display results
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(avg_filter)
plt.title("Averaging Filter")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(gaussian_filter)
plt.title("Gaussian Filter")
plt.axis("off")

plt.subplot(2, 3, 4)
```

```
plt.imshow(median_filter)
plt.title("Median Filter")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(bilateral_filter)
plt.title("Bilateral Filter")
plt.axis("off")

plt.tight_layout()
plt.show()

# Save filtered images
cv2.imwrite("average_filter.jpg", cv2.cvtColor(avg_filter,
cv2.COLOR_RGB2BGR))
cv2.imwrite("gaussian_filter.jpg", cv2.cvtColor(gaussian_filter,
cv2.COLOR_RGB2BGR))
cv2.imwrite("median_filter.jpg", cv2.cvtColor(median_filter,
cv2.COLOR_RGB2BGR))
cv2.imwrite("bilateral_filter.jpg", cv2.cvtColor(bilateral_filter,
cv2.COLOR_RGB2BGR))
```




```
True
```

Practical 7

```
import cv2
import matplotlib.pyplot as plt
```

```
# Read image
img = cv2.imread("download.png")

# Convert BGR to RGB for displaying with matplotlib
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Apply Blur
blurred_img = cv2.blur(img_rgb, (15, 15))

# Display Original and Blurred Images
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(blurred_img)
plt.title("Blurred Image")
plt.axis("off")

plt.show()

# Save blurred image
cv2.imwrite("blurred_image.jpg", cv2.cvtColor(blurred_img,
cv2.COLOR_RGB2BGR))
```


```
True
```

Practical 8

```
import cv2

# Read image
image = cv2.imread("download.png")
```

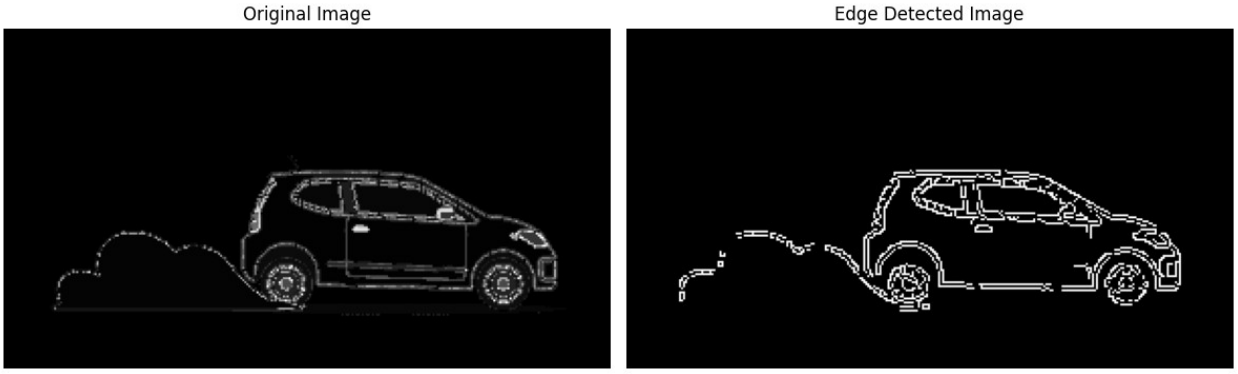
```
height, width, channels = image.shape

print("Image Metadata")
print("----------------")
print("Width       :", width, "pixels")
print("Height      :", height, "pixels")
print("Channels    :", channels)
print("Data Type   :", image.dtype)
print("Image Size  :", image.size, "pixels")
print("Shape       :", image.shape)
```

```
Image Metadata
----------------
Width       : 300 pixels
Height      : 168 pixels
Channels    : 3
Data Type   : uint8
Image Size  : 151200 pixels
Shape       : (168, 300, 3)
```

Practical 9

```
from PIL import Image
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Open image using Pillow
img = Image.open("download.png")

# Convert image to NumPy array
img_np = np.array(img)

# Check image channels and convert to grayscale if needed
if len(img_np.shape) == 2:
    # Image is already grayscale
    gray = img_np
elif img_np.shape[2] == 4:
    # RGBA image
    gray = cv2.cvtColor(img_np, cv2.COLOR_RGBA2GRAY)
else:
    # RGB image
    gray = cv2.cvtColor(img_np, cv2.COLOR_RGB2GRAY)

# Apply Canny Edge Detection
edges = cv2.Canny(gray, 100, 200)

# Display Original Image
plt.figure(figsize=(12, 6))
```

```
plt.subplot(1, 2, 1)
if len(img_np.shape) == 2:
    plt.imshow(img_np, cmap='gray')
else:
    plt.imshow(img_np)
plt.title("Original Image")
plt.axis("off")

# Display Edge Detected Image
plt.subplot(1, 2, 2)
plt.imshow(edges, cmap='gray')
plt.title("Edge Detected Image")
plt.axis("off")

plt.tight_layout()
plt.show()

# Save edge-detected image
cv2.imwrite("edge_detected.jpg", edges)

print("Edge detection completed successfully!")
print("Output saved as 'edge_detected.jpg'")
```




```
Edge detection completed successfully!
Output saved as 'edge_detected.jpg'
```